

Reviewer Pack — Noncommutative Fourier Coefficient Norm Separates Read-Twice...

Entry 3dbeb5b4ba3d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 00:21:54 UTC

Noncommutative Fourier Coefficient Norm Separates Read-Twice from Read-Once BPs

Entry ID: 3dbeb5b4ba3d **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 00:21:54 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): Communication Complexity Lower Bounds for Read-Twice Branching Programs

Statement:

For any read-once BP P , the norm of its noncommutative Fourier coefficients is $O(\log n)$, while for any read-twice BP P , it is $\Omega(n)$. This separation implies that distinguishing between read-once and read-twice BPs requires communication complexity $\Omega(n)$.

Rationale (proposer’s reasoning):

Noncommutative harmonic analysis captures the non-abelian structure of read-twice BPs via group representations, which read-once BPs lack. The exponential growth of the norm for read-twice BPs reflects their inherent complexity, enabling Fourier-analytic lower bounds.

Taxonomy category: BP_READTWICE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2384bcfa0b06d9a8

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	UNCERTAIN	0.95	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=-9, elapsed=82.4s

5.1 Generated Python source

```
import random
import math
from itertools import permutations

def generate_random_cnf(n):
    clauses = []
    for _ in range(n):
        clause = [random.randint(1, n), -random.randint(1, n)]
        clauses.append(clause)
    return clauses

def communication_matrix(CNF):
    n = len(CNF)
    C = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            if any(i in clause and -j in clause for clause in CNF):
                C[i][j] = 1
    return C

def symmetric_group_representations(n):
    G = list(permutations(range(1, n + 1)))
    return G

def noncommutative_fourier_coefficients(CNF, G):
    n = len(CNF)
    F = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(1, n + 1):
        for j in range(1, n + 1):
            F[j][i] = sum(CNF[j - 1][g[i - 1]] for g in G) / len(G)
    return F

def norm_of_matrix(M):
    max_norm = 0
    for row in M:
        row_norm = sum(abs(x) for x in row)
        if row_norm > max_norm:
            max_norm = row_norm
    return max_norm
```

```

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    CNF = generate_random_cnf(n)
    C = communication_matrix(CNF)
    G = symmetric_group_representations(n)
    F = noncommutative_fourier_coefficients(CNF, G)
    norm_F = norm_of_matrix(F)

    if n <= 2:
        return {
            "metric_name": "norm",
            "metric_value": norm_F,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    is_read_once = all(C[i][j] == 0 for i in range(1, n + 1) for j in range(i + 1, n + 1))
    conjecture_holds = (is_read_once and norm_F <= math.log(n)) or (not is_read_once and norm_F >= math.log(n))

    return {
        "metric_name": "norm",
        "metric_value": norm_F,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or list(range(2, 30)) + [random.randint(100, 999) for _ in range(10)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        counterexample = "mapping_undefined"
        print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

(empty)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed before producing data, preventing evaluation of support fraction or counterexamples. | next: Re-run test with detailed logging to identify crash cause and collect empirical results

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	83814
2	preregistration	ollama_remote	qwen3:8b	0	0	23976
3	novelty	ollama_remote	qwen3:8b	0	0	20753
4	novelty	ollama_remote	qwen3:8b	0	0	14704
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17191
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10608
7	judge	ollama_remote	qwen3:8b	0	0	120982

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 292029 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in *research/audit/3dbeb5b4ba3d.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/3dbeb5b4ba3d.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/3dbeb5b4ba3d.tar.gz* (if generated)